



TEMA 1: CREACIÓN DE JOBS Y ALERTS CON SQL SERVER AGENT

Automatiza tareas, genera reportes y recibe alertas para mantener tu base de datos siempre bajo control.



PROYECTO 1 REPORTE DIARIO DE TRÁMITES

ENUNCIADO

La Oficina de Grados y Títulos requiere que todos los días a las 08:00 AM se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.

DEBES:

- ✓ Crear un Job
- ✓ Ejecutar un script T-SQL
- ✓ Programarlo diariamente
- ✓ Registrar el resultado en una tabla de auditoría



SCRIPT / SOLUCIÓN

1. Tabla de auditoría

```
CREATE TABLE AuditoriaJobs(  
  Id INT IDENTITY PRIMARY KEY,  
  Fecha DATETIME DEFAULT GETDATE(),  
  TotalTramites INT  
);
```

2. Script del Job

```
INSERT INTO AuditoriaJobs(TotalTramites)  
SELECT COUNT(*)  
FROM Trámite  
WHERE CAST(FechaRegistro AS  
DATE)=CAST(GETDATE()-1 AS DATE);
```

3. Crear Job

```
EXEC sp_add_job  
@job_name='Reporte Diario Tramites';
```

4. Paso del Job (T-SQL)

```
EXEC sp_add_jobstep  
@job_name='Reporte Diario Tramites',  
@step_name='Contar Tramites',  
@subsystem='TSQL',  
@command='  
INSERT INTO AuditoriaJobs(TotalTramites)  
SELECT COUNT(*)  
FROM Trámite  
WHERE CAST(FechaRegistro AS DATE)=  
CAST(GETDATE()-1 AS DATE)';
```

5. Programación diaria a las 08:00 AM

```
EXEC sp_add_schedule  
@schedule_name='Diario0800',  
@freq_type=4, -- diaria  
@active_start_time=080000;
```



JUSTIFICACIÓN

El Job automatiza la generación del reporte diario evitando procesos manuales y permitiendo un control histórico de los resultados.



BUENAS PRÁCTICAS

- Registrar siempre la ejecución del Job.
- Programar fuera de horas pico.
- Verificar el historial del Job en SQL Server Agent.
- Mantener la tabla de auditoría sin crecer innecesariamente.



PROYECTO 2 ACTUALIZACIÓN SEMANAL DE TRÁMITES

ENUNCIADO

Actualizar semanalmente el campo FechaUltMod para todos los trámites en estado EN_PROCESO y registrar la ejecución.



SCRIPT / SOLUCIÓN

1. Script de actualización

```
UPDATE Trámite  
SET FechaUltMod=GETDATE()  
WHERE Estado='EN_PROCESO';
```

2. Crear Job

```
EXEC sp_add_job  
@job_name='Actualizar Fecha Tramites';
```

3. Programación semanal

```
EXEC sp_add_schedule  
@schedule_name='Semanal',  
@freq_type=8; -- semanal
```



JUSTIFICACIÓN

Automatiza la actualización de la fecha de modificación sin intervención del administrador, manteniendo la información al día.



BUENAS PRÁCTICAS

- Registrar la fecha de ejecución.
- Revisar historial del Job.
- Ejecutar en horarios de baja actividad.
- Verificar que los registros se actualicen correctamente.

PROYECTO 3 LIMPIEZA MENSUAL DE REGISTROS TEMPORALES

ENUNCIADO

Eliminar mensualmente los registros temporales de la tabla TrámiteLogTemporal. Los registros mayores a 180 días deben almacenarse previamente en una tabla histórica.



1. Crear tabla histórica

```
SELECT * INTO HistoricoTrámite  
FROM TrámiteLogTemporal  
WHERE l>180;
```

2. Copiar históricos (>180 días)

```
INSERT INTO HistoricoTrámite  
SELECT *  
FROM TrámiteLogTemporal  
WHERE Fecha=DATEADD(DAY,-180,GETDATE());
```

3. Eliminar registros antiguos

```
DELETE  
FROM TrámiteLogTemporal  
WHERE Fecha=DATEADD(DAY,-180,GETDATE());
```

4. Todo en transacción

```
BEGIN TRY  
BEGIN TRANSACTION  
INSERT INTO HistoricoTrámite  
SELECT *  
FROM TrámiteLogTemporal  
WHERE Fecha=DATEADD(DAY,-180,GETDATE());  
  
DELETE  
FROM TrámiteLogTemporal  
WHERE Fecha=DATEADD(DAY,-180,GETDATE());  
COMMIT;  
END TRY  
BEGIN CATCH  
ROLLBACK;  
END CATCH;
```



JUSTIFICACIÓN

Evita el crecimiento excesivo de la tabla temporal y conserva la información histórica en una tabla de respaldo.



BUENAS PRÁCTICAS

- Respalda antes de eliminar.
- Ejecutar durante la madrugada.
- Usar transacción para asegurar la integridad.
- Verificar el número de registros copiados y eliminados.

PROYECTO 4 ALERT PARA ERRORES CRÍTICOS

ENUNCIADO

Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a 17 durante el registro de trámites.



SCRIPT / SOLUCIÓN

1. Crear alerta

```
EXEC msdb.dbo.sp_add_alert  
@name='Errores Críticos',  
@severity=17;
```

2. Asociar operador para notificación

```
EXEC msdb.dbo.sp_add_notification  
@alert_name='Errores Críticos',  
@operator_name='Administrador',  
@notification_method=1;
```



JUSTIFICACIÓN

Permite detectar y notificar rápidamente errores críticos para actuar de inmediato y evitar afectaciones mayores.



BUENAS PRÁCTICAS

- Configurar correctamente los operadores.
- Probar el envío de notificaciones.
- Mantener Database Mail configurado.
- Revisar la severidad adecuada (>=17).

PROYECTO 5 REPORTE DIARIO DE TRÁMITES PENDIENTES

ENUNCIADO

Implementar un sistema automático que diariamente:

- Verifique trámites pendientes mayores a 30 días.
- Genere un reporte.
- Registre la ejecución.
- Envíe un correo mediante Database Mail.
- Almacene estadísticas de ejecución.



SCRIPT / SOLUCIÓN

1. Trámites pendientes > 30 días

```
SELECT *  
FROM Trámite  
WHERE DATEDIFF(DAY, FechaRegistro,  
GETDATE())>30;
```

2. Registrar ejecución

```
INSERT INTO AuditoriaJobs(TotalTramites)  
SELECT COUNT(*)  
FROM Trámite  
WHERE DATEDIFF(DAY, FechaRegistro,  
GETDATE())>30;
```

3. Enviar correo con Database Mail

```
EXEC msdb.dbo.sp_send_dbmail  
@profile_name='PerfilCorreo',  
@recipients='admin@universidad.edu',  
@subject='Reporte Trámites Pendientes',  
@body='Se adjunta el reporte de trámites  
pendientes mayores a 30 días.';
```

4. Programar Job diario

```
EXEC sp_add_job @job_name='Reporte Pendientes 30 días';  
-- Agregar el paso con el script anterior  
-- Programar con sp_add_schedule (diario)
```



JUSTIFICACIÓN

El sistema automatiza el control de trámites pendientes y notifica oportunamente a los responsables, mejorando la gestión.



BUENAS PRÁCTICAS

- Verificar configuración de Database Mail.
- Validar que el correo se envíe correctamente.
- Revisar historial del Job y corregir fallos.
- Programar en horarios de baja carga.
- Mantener registros de auditoría para seguimiento.



SQL SERVER AGENT

Herramienta que permite programar tareas automáticas (Jobs), definir alertas y administrar notificaciones para mantener el servidor y las bases de datos bajo control.

COMPONENTES CLAVE

- Jobs: Tareas programadas.
- Alerts: Notificaciones de eventos.
- Operators: Destinatarios de alertas.
- History: Historial de ejecución.

BENEFICIOS

- ✓ Automatización de tareas repetitivas.
- ✓ Mayor control y monitoreo del sistema.
- ✓ Detección temprana de errores críticos.
- ✓ Mejora en la disponibilidad y rendimiento.
- ✓ Historial y auditoría de ejecuciones.

BUENAS PRÁCTICAS GENERALES

- Utilizar nombres descriptivos para Jobs, Schedules y Alerts.
- Documentar los Jobs y su propósito.
- Revisar periódicamente el historial de ejecución.
- Mantener la base de datos y estadísticas actualizadas.
- Hacer pruebas antes de poner en producción.

FLUJO GENERAL



AUTOMATIZAR ES AHORRAR TIEMPO Y GARANTIZAR CALIDAD EN LA INFORMACIÓN





TEMA 2: USO DE DATABASE MAINTENANCE PLANS

Automatiza el mantenimiento de tu base de datos para mejorar el rendimiento, la disponibilidad y la integridad de la información.



¿QUÉ ES?

Los Maintenance Plans permiten automatizar tareas periódicas de mantenimiento para que la base de datos funcione de forma óptima sin intervención manual.



TAREAS CLAVE QUE SE PUEDEN AUTOMATIZAR



Backup



Verificación (CheckDB)



Reorganización de índices



Actualización de estadísticas



Limpieza del historial



Programación de tareas

PROYECTO 1

PLAN DIARIO DE RESPALDO COMPLETO

ENUNCIADO

Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de la base de datos GradosTítulos.



SOLUCIÓN / SCRIPT

Tareas del plan:

- Backup Database (Completo)
- Compresión: SI
- Verificación: SI
- Programación: Diaria 02:00 AM

SCRIPT (T-SQL equivalente)

```
BACKUP DATABASE [GradosTítulos]
TO DISK = 'D:\Backups\GradosTítulos_'
+ CONVERT(VARCHAR(8), GETDATE(), 112)
+ '.bak'
WITH INIT, COMPRESSION;
GO
```



JUSTIFICACIÓN

Permite recuperar la base de datos en caso de fallos, pérdidas de datos o errores humanos.

BUENAS PRÁCTICAS

- Guardar los backups en otra unidad o servidor.
- Verificar periódicamente que los backups se generen correctamente.
- Mantener copias de respaldo históricas según la política de retención.

PROYECTO 2

REORGANIZAR ÍNDICES SEMANALMENTE

ENUNCIADO

Crear un plan semanal para reorganizar índices de las tablas:

- Persona
- Tramite
- TrabajoInvestigacion
- Sustentacion

SOLUCIÓN / SCRIPT

Tareas del plan:

- Reorganize Index (Todas las tablas)
- Seleccionar tablas específicas
- Umbral: Fragmentación > 5 %
- Programación: Semanal (Domingo 01:00 AM)

SCRIPT (T-SQL equivalente)

```
ALTER INDEX ALL ON [Persona] REORGANIZE;
ALTER INDEX ALL ON [Tramite] REORGANIZE;
ALTER INDEX ALL ON [TrabajoInvestigacion] REORGANIZE;
ALTER INDEX ALL ON [Sustentacion] REORGANIZE;
GO
```



JUSTIFICACIÓN

Mejora el rendimiento de las lecturas al reducir la fragmentación de los índices sin afectar su disponibilidad.



BUENAS PRÁCTICAS

- Usar REORGANIZE para fragmentación < 30 %.
- Usar REBUILD para fragmentación > 30 %.
- Programar en horarios de baja actividad.

PROYECTO 3

PLAN DE MANTENIMIENTO PERSONALIZADO

ENUNCIADO

Diseñar un plan que:

- Reorganice índices menores al 30 %
- Reconstruya índices mayores al 30 %
- Actualice estadísticas

SOLUCIÓN / SCRIPT

Tareas del plan:

- Reorganize Index (Fragmentación < 30 %)
- Rebuild Index (Fragmentación > 30 %)
- Update Statistics
- Programación: Semanal (Sábado 02:00 AM)

SCRIPT (T-SQL equivalente)

```
-- Reorganizar índices menores al 30 %
SELECT name FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED')
WHERE avg_fragmentation_in_percent BETWEEN 5 AND 30;

-- Reconstruir índices mayores al 30 %
SELECT name FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED')
WHERE avg_fragmentation_in_percent > 30;

-- Actualizar estadísticas
UPDATE STATISTICS GradosTítulos;
GO
```



JUSTIFICACIÓN

Mantiene los índices y estadísticas optimizados para consultas más rápidas y eficientes.

BUENAS PRÁCTICAS

- Revisar el historial del plan después de cada ejecución.
- Establecer umbrales adecuados según el tamaño de la BD.
- Actualizar estadísticas regularmente.

PROYECTO 4

MANTENIMIENTO COMPLETO DE LA BASE DE DATOS

ENUNCIADO

Automatizar el mantenimiento completo de la base de datos incluyendo:

- Backup
- CHECKDB
- Limpieza del historial
- Reorganización de índices
- Actualización de estadísticas

SOLUCIÓN / SCRIPT

Tareas del plan:

- Backup Database (Completo)
- Check Database Integrity (CHECKDB)
- Purge History (Limpieza del historial)
- Reorganize / Rebuild Index
- Update Statistics
- Programación: Diaria 01:00 AM

SCRIPT (T-SQL comprobaciones)

```
-- Verificar integridad
DBCC CHECKDB ('GradosTítulos') WITH NO_INFOMSGS;

-- Limpiar historial de mantenimiento (ejemplo 30 días)
EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = DATETIME(DAY, -30, GETDATE());
GO
```



JUSTIFICACIÓN

Asegura la integridad de la base de datos y mantiene su rendimiento de forma continua.



BUENAS PRÁCTICAS

- Mantener backups antes de realizar operaciones.
- Verificar alertas después de cada ejecución.
- No ejecutar tareas pesadas en horas pico.

PROYECTO 5

PLAN INTELIGENTE DE MANTENIMIENTO

ENUNCIADO

Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos.



SOLUCIÓN / SCRIPT

Tareas del plan:

- Ejecutar cursores para evaluar fragmentación
- Decidir entre REORGANIZE o REBUILD
- Actualizar estadísticas
- Registrar en tabla de auditoría
- Programación: (Viernes 01:30 AM)

SCRIPT (T-SQL equivalente)

```
DECLARE @db sysname = 'GradosTítulos', @IndexName sysname,
        @frag FLOAT, @sql NVARCHAR(MAX);

DECLARE cur CURSOR FOR
SELECT QUOTENAME(OBJECT_SCHEMA_NAME(object_id)) + '.' + QUOTENAME(OBJECT_NAME(object_id)) AS Tabla,
       name, avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(@db), NULL, NULL, NULL, 'SAMPLED')
WHERE avg_fragmentation_in_percent > 5;

OPEN cur; FETCH NEXT FROM cur INTO @IndexName, @sql, @frag;
WHILE @@FETCH_STATUS = 0
BEGIN
    SET @sql = CASE WHEN @frag BETWEEN 5 AND 30 THEN 'ALTER INDEX [' + @sql + '] ON ' + @IndexName + ' REORGANIZE;'
                  ELSE 'ALTER INDEX [' + @sql + '] ON ' + @IndexName + ' REBUILD;'
    END;
    EXEC (@sql);
    FETCH NEXT FROM cur INTO @IndexName, @sql, @frag;
END;
CLOSE cur; DEALLOCATE cur;

UPDATE STATISTICS GradosTítulos;
GO
```



JUSTIFICACIÓN

El plan decide automáticamente la mejor acción (REORGANIZE o REBUILD) según el nivel de fragmentación, optimizando el tiempo y los recursos.

BUENAS PRÁCTICAS

- Probar el plan primero en un entorno de pruebas.
- Registrar todas las acciones realizadas.
- Ajustar los umbrales de fragmentación según la carga del sistema.



TABLA DE AUDITORÍA (Ejemplo)

```
CREATE TABLE AuditoriaMantenimiento(
  Id INT IDENTITY PRIMARY KEY,
  FechaEjecucion DATETIME DEFAULT GETDATE(),
  Operacion NVARCHAR(50),
  Detalles NVARCHAR(2000)
);
```



BENEFICIOS

- Mejora el rendimiento de consultas.
- Reduce la fragmentación de índices.
- Asegura la integridad de los datos.
- Minimiza tiempos de inactividad.
- Automatiza tareas críticas.



RECOMENDACIONES GENERALES

- Revisar periódicamente los Maintenance Plans y su historial.
- Validar que los backups se generen y restauren correctamente.
- Programar tareas en horarios de baja actividad.
- Monitorear alertas y notificaciones del SQL Server Agent.



HERRAMIENTAS RELACIONADAS

- SQL Server Agent
- Database Mail
- Performance Monitor
- SQL Server Management Studio (SSMS)



TEMA 3: AUTOMATIZACIÓN CON T-SQL Y POWERSHELL

Automatiza procesos en SQL Server combinando T-SQL y PowerShell para ganar eficiencia, seguridad y control.



¿QUÉ ES LA AUTOMATIZACIÓN?

Uso de scripts y herramientas para ejecutar tareas repetitivas sin intervención manual, asegurando consistencia, trazabilidad y ahorro de tiempo.



BENEFICIOS



PROYECTO 1: PROCEDIMIENTO ALMACENADO PARA RESPALDOS

ENUNCIADO

Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente lo ejecute mediante PowerShell.



T-SQL (Procedimiento)

```
CREATE PROCEDURE sp_RespaldoCompleto
AS
BEGIN
    DECLARE @fecha VARCHAR(8) =
        CONVERT(VARCHAR(8), GETDATE(), 112);
    DECLARE @ruta VARCHAR(200) =
        'C:\Respaldos\BD_' + @fecha + '.bak';
    BACKUP DATABASE GradosTitulos
    TO DISK = @ruta
    WITH INIT, COMPRESSION, STATS = 10;
END;
```

PowerShell (Ejecución)

```
Invoke-Sqlcmd -ServerInstance
"SQLSERVER"
-Database 'GradosTitulos'
-Query "EXEC sp_RespaldoCompleto"

Ejecutar script desde PowerShell:
.\RespaldoBD.ps1
```



JUSTIFICACIÓN

Permite generar respaldos de forma automática y centralizada asegurando la disponibilidad de la información.

BUENAS PRÁCTICAS

- Guardar respaldos en otra unidad o servidor.
- Comprimir los respaldos para ahorrar espacio.
- Verificar la integridad del respaldo periódicamente.
- Conservar respaldos según la política de retención.

PROYECTO 2: EXPORTAR CLIENTES A CSV CON POWERSHELL

ENUNCIADO

Automatizar la exportación diaria de los clientes aprobados hacia un archivo CSV utilizando PowerShell.



PowerShell

```
$servidor = "SQLSERVER"
$dbd = "GradosTitulos"
$consulta = "SELECT IdCliente, Nombre, Email, FechaAprobacion
FROM Cliente WHERE Estado = 'Aprobado'"
$fecha = Get-Date -Format "yyyyMMdd"
$ruta = "C:\Exportaciones\Clientes_Aprobados_$fecha.csv"

Invoke-Sqlcmd -ServerInstance $servidor -Database $dbd -Query $consulta |
Export-Csv -Path $ruta -NoTypeInformation -Encoding UTF8
```



JUSTIFICACIÓN

Extrae información de manera automática para análisis o intercambio de datos.

BUENAS PRÁCTICAS

- Programar la tarea en el Programador de Windows.
- Definir rutas y nombres dinámicos por fecha.
- Validar que el archivo CSV se genere correctamente.
- Asegurar permisos de escritura en la carpeta destino.

PROYECTO 3: PROCESO AUTOMÁTICO DE TRÁMITES

ENUNCIADO

Desarrollar un proceso que:

- consulte la cantidad de trámites
- genere un archivo Excel
- almacene una copia en una carpeta de respaldo.



T-SQL (Consulta)

```
SELECT COUNT(*) AS TotalTramites
FROM Trámite;
```

PowerShell (Exportar a Excel)

```
$servidor = "SQLSERVER"
$dbd = "GradosTitulos"
$consulta = "SELECT * FROM Trámite"
$datos = Invoke-Sqlcmd -ServerInstance $servidor
-Database $dbd -Query $consulta
$fecha = Get-Date -Format "yyyyMMdd"
$rutaExcel = "C:\Reportes\Tramites_$fecha.xlsx"
$datos | Export-Excel -Path $rutaExcel -AutoSize
```

PowerShell (Copia de respaldo)

```
Copy-Item $rutaExcel -Destination "D:\Backups\Reportes\$fecha"
```



JUSTIFICACIÓN

Permite generar reportes automáticos para control y análisis, además de mantener copias de seguridad.

BUENAS PRÁCTICAS

- Validar que la carpeta exista antes de copiar.
- Usar rutas con fecha para evitar sobrescritura.
- Comprobar la generación del archivo.

PROYECTO 4: DETECTAR INACTIVOS Y GENERAR REPORTE

ENUNCIADO

Crear un sistema que detecte docentes inactivos por más de un año y genere un reporte automáticamente.



T-SQL (Consulta)

```
SELECT IdDocente, Nombre, FechaUltAcceso
FROM Docente
WHERE FechaUltAcceso < DATEADD(YEAR, -1, GETDATE());
```

PowerShell (Generar reporte)

```
$servidor = "SQLSERVER"
$dbd = "GradosTitulos"
$consulta = "SELECT IdDocente, Nombre, FechaUltAcceso
FROM Docente
WHERE FechaUltAcceso < DATEADD(YEAR, -1, GETDATE())"
$datos = Invoke-Sqlcmd -ServerInstance $servidor -Database $dbd -Query $consulta
$fecha = Get-Date -Format "yyyyMMdd"
$ruta = "C:\Reportes\Inactivos_$fecha.csv"
$datos | Export-Csv -Path $ruta -NoTypeInformation -Encoding UTF8
```



JUSTIFICACIÓN

Identifica usuarios inactivos para tomar acciones administrativas o de depuración.

BUENAS PRÁCTICAS

- Programar la ejecución mensual.
- Almacenar históricos de reportes.
- Notificar al responsable mediante correo (opcional).

PROYECTO 5: SOLUCIÓN DE AUTOMATIZACIÓN COMPLETA

ENUNCIADO

Implementar una solución completa que:

- ejecute respaldos
- valide con CHECKDB
- genere reportes
- comprima respaldos
- elimine respaldos mayores a 30 días.



PowerShell (Solución completa)

```
$servidor = "SQLSERVER"
$dbd = "GradosTitulos"
$fecha = Get-Date -Format "yyyyMMdd"
$rutaBackup = "C:\Respaldos\BD_$fecha.bak"
$rutaZip = "C:\Respaldos\BD_$fecha.zip"
$rutaLog = "C:\Logs\Mantenimiento_$fecha.log"

# 1. Respaldo completo
Invoke-Sqlcmd -ServerInstance $servidor -Query "BACKUP DATABASE $dbd TO DISK = '$rutaBackup' WITH INIT, COMPRESSION" |
Out-File -FilePath $rutaLog -Append

# 2. Validar integridad
Invoke-Sqlcmd -ServerInstance $servidor -Query "DBCC CHECKDB ('$dbd') WITH NO_INFOMSGS" |
Out-File -FilePath $rutaLog -Append

# 3. Mantener log
"mantenimiento ejecutado el: $fecha" | Out-File -FilePath $rutaLog -Append

# 4. Comprimir respaldo
Compress-Archive -Path $rutaBackup -DestinationPath $rutaZip -Force

Get-Childitem -Path "C:\Respaldos" -Filter "*.zip" |
Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-30) } |
Remove-Item -Force
```



JUSTIFICACIÓN

Automatiza de punta a punta el mantenimiento de la base de datos, asegurando integridad, seguridad y trazabilidad.



BUENAS PRÁCTICAS

- Probar en un entorno controlado primero.
- Verificar permisos y rutas de acceso.
- Gestionar permisos con Try/Catch (opcional).
- Revisar logs y espacios en disco.
- Monitorear la ejecución periódicamente.



MANEJO DE ERRORES (Ejemplo básico)

```
try {
    # aquí va el código principal
} catch {
    "Error: $_" | Out-File -FilePath $rutaLog -Append
}
```



HERRAMIENTAS UTILIZADAS

- SQL Server (T-SQL)
- PowerShell
- SQL Server Management Studio (SSMS)
- Programador de tareas de Windows



RECOMENDACIONES GENERALES

- Documentar todos los procesos y scripts.
- Mantener las rutas y nombres estandarizados.
- Realizar respaldos antes de cambios importantes.
- Automatizar horarios en horas de baja actividad.
- Revisar logs y alertas regularmente.



BENEFICIOS FINALES

- Mayor disponibilidad de la información.
- Menor intervención manual y menos errores.
- Mejor rendimiento y administración.
- Cumplimiento de políticas de respaldo y auditoría.
- Toma de decisiones basada en reportes confiables.



TEMA 4: MONITOREO PROACTIVO (CORREO, LOGS Y ALERTAS)

Supervisa tu base de datos, detecta problemas a tiempo y recibe alertas automáticas.



¿QUÉ ES?

El monitoreo proactivo permite controlar el rendimiento, registrar eventos y recibir alertas para resolver problemas antes de que afecten a los usuarios.



Detección temprana



Mayor estabilidad



Mejor rendimiento



Alertas por correo



Historial y auditoría



Decisiones informadas

BENEFICIOS CLAVE

1 CONSULTA DIARIA DE TRÁMITES PENDIENTES Y ENVÍO DE RESUMEN

ENUNCIADO

Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico con el resumen.



SCRIPT / SOLUCIÓN

```
CREATE PROCEDURE SP_ResumenTramitesPendientes
AS
BEGIN
    DECLARE @Total INT
    SELECT @Total = COUNT(*)
    FROM Trámite
    WHERE Estado = 'PENDIENTE';

    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'PerfilCorreo',
        @recipients = 'admin@universidad.edu',
        @subject = 'Resumen Diario de Trámites',
        @body = 'Total de trámites pendientes: ' +
            CAST(@Total AS VARCHAR(20));
END
```



JUSTIFICACIÓN
Permite conocer la situación diaria de trámites pendientes sin ingresar manualmente.



BUENAS PRÁCTICAS

- ✓ Validar Database Mail.
- ✓ Usar perfiles dedicados.
- ✓ Programar en horas de baja carga.

2 REGISTRO AUTOMÁTICO DE ERRORES

ENUNCIADO

Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento Trámite.SP_RegistrarTrámite.



SCRIPT / SOLUCIÓN

```
CREATE TABLE AuditoriaErrores (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATETIME DEFAULT GETDATE(),
    Procedimiento NVARCHAR(100),
    NumeroError INT,
    MensajeError NVARCHAR(4000)
);

ALTER PROCEDURE Trámite.SP_RegistrarTrámite
AS
BEGIN TRY
    -- Lógica del procedimiento
END TRY
BEGIN CATCH
    INSERT INTO AuditoriaErrores
        (Procedimiento, NumeroError, MensajeError)
    VALUES ('Trámite.SP_RegistrarTrámite',
        ERROR_NUMBER(), ERROR_MESSAGE());
    THROW; -- Re-lanza el error
END CATCH
END
```



JUSTIFICACIÓN
Permite llevar un control histórico de los errores para análisis y mejora continua.



BUENAS PRÁCTICAS

- ✓ Registrar fecha y hora.
- ✓ Capturar número y mensaje del error.
- ✓ Usar TRY...CATCH en procedimientos críticos.

3 ALERTA POR CRECIMIENTO EXCESIVO DEL LOG

ENUNCIADO

Diseñar un sistema que monitoree el crecimiento del archivo de log y genere una alerta cuando supere el 80 % del tamaño configurado.



SCRIPT / SOLUCIÓN

```
-- Consulta de uso del log
SELECT DB_NAME(database_id) AS BaseDatos,
       used_log_space_in_percent AS Usolog
FROM sys.dm_db_log_space_usage;

-- Alerta (SQL Server Agent)
EXEC msdb.dbo.sp_add_alert
    @name = 'Log al 80%',
    @message_id = 0,
    @severity = 16,
    @enabled = 1;

EXEC msdb.dbo.sp_add_notification
    @alert_name = 'Log al 80%',
    @operator_name = 'DBA',
    @notification_method = 1; -- E-mail
```



JUSTIFICACIÓN
Evita que el log llene el disco y cause interrupciones en el servicio.



BUENAS PRÁCTICAS

- ✓ Revisar el crecimiento regularmente.
- ✓ Configurar alertas con umbrales adecuados.
- ✓ Revisar historial de alertas.

4 TIEMPO DE EJECUCIÓN Y RANKING DE PROCEDIMIENTOS

ENUNCIADO

Registrar automáticamente el tiempo de ejecución de los procedimientos almacenados principales y generar un ranking de rendimiento.



SCRIPT / SOLUCIÓN

```
CREATE TABLE MonitoreoTiempo (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Procedimiento NVARCHAR(100),
    TiempoInicio DATETIME,
    TiempoFin DATETIME,
    DuracionMs INT
);

-- Ejemplo de captura (al inicio y fin del proc)
INSERT INTO MonitoreoTiempo(Procedimiento, TiempoInicio)
VALUES ('Trámite.SP_RegistrarTrámite', GETDATE());

-- Al finalizar
UPDATE MonitoreoTiempo SET TiempoFin = GETDATE(),
    DuracionMs = DATEDIFF(MILLISECOND, TiempoInicio, GETDATE())
WHERE Id = SCOPE_IDENTITY();

-- Ranking de rendimiento
SELECT TOP 10 Procedimiento,
    AVG(DuracionMs) AS PromedioMs
FROM MonitoreoTiempo
GROUP BY Procedimiento
ORDER BY PromedioMs DESC;
```



JUSTIFICACIÓN
Permite identificar procedimientos lentos y optimizar el rendimiento de la base de datos.



BUENAS PRÁCTICAS

- ✓ Registrar solo procedimientos críticos.
- ✓ Limpiar registros antiguos periódicamente.
- ✓ Usar índices en la tabla de monitoreo.

5 SISTEMA INTEGRAL DE MONITOREO Y ALERTAS

ENUNCIADO

Desarrollar un sistema integral que:

- Monitoree bloqueos
- Detecte consultas lentas
- Identifique alto consumo de CPU
- Registre eventos históricos
- Envíe alertas por correo
- Genere informe diario del estado del servidor



SCRIPT / SOLUCIÓN (Componentes principales)

```
1. Bloqueos (Blocking)
SELECT blocking_session_id, session_id,
       wait_type, wait_time_ms, text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle)
WHERE blocking_session_id <> 0;

2. Consultas lentas
SELECT TOP 20 B
    session_id, total_cpu_time/1000 AS CPU,
    text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle)
ORDER BY total_cpu_time DESC;

3. Alto uso (Paging)
SELECT TOP 20
    session_id, total_elapsed_time/1000 AS TiempoSeg,
    execution_count, text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle)
ORDER BY total_elapsed_time DESC;
```

```
4. Enviar reporte por correo
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'PerfilCorreo',
    @recipients = 'admin@universidad.edu',
    @subject = 'Informe Diario del Servidor',
    @body = 'Se adjunta el reporte diario.',
    @file_attachments = 'C:\Reportes\Informe.xlsx';

5. Programación (SQL Server Agent)
Programar el job diario para ejecutar
el script y enviar el informe.
```



JUSTIFICACIÓN
Brinda visibilidad completa del estado del servidor y permite acciones preventivas.



BUENAS PRÁCTICAS

- ✓ Consolidar toda la información en un solo reporte.
- ✓ Usar Database Mail con perfil dedicado.
- ✓ Revisar y ajustar umbrales.
- ✓ Validar logs y eventos periódicamente.

HERRAMIENTAS UTILIZADAS

- T-SQL (Transact-SQL)
- PowerShell
- SQL Server Agent
- Database Mail
- DMV (Dynamic Management Views)
- Extended Events / SQL Trace



BUENAS PRÁCTICAS GENERALES

- ✓ Monitorear constantemente el rendimiento.
- ✓ Documentar procesos y mantener auditorías.
- ✓ Configurar alertas con umbrales realistas.
- ✓ Automatizar tareas repetitivas.
- ✓ Revisar y mantener los jobs y alertas.

FLUJO DEL SISTEMA DE MONITOREO



¡Menos problemas, más control y mejor rendimiento para tu base de datos!